ARIMA NOTEBOOKS

OPEN-SOURCE · LOCAL-FIRST · MULTI-LANGUAGE

PRODUCT BROCHURE · V3.1

Arima Notebooks

Interactive multi-language notebooks in your browser — **Java, JShell, JavaScript, TypeScript, C#, F#, C++** — with first-class AI assistance from Claude, GitHub Copilot, and Gemini. Runs entirely on your machine. No cloud account. No telemetry.

JShell · Java · JavaScript · TypeScript · C# · F# · C++

An open-source product by **Suresh Chande** — founding contributor

github.com/snchande/arima-notebooks

01 What is Arima?

Arima Notebooks is a **locally-hosted, browser-based notebook environment** for seven languages — JShell, Java, JavaScript, TypeScript, C#, F#, and C++ — with three AI co-pilots (Claude, GitHub Copilot, Gemini) wired in as **local CLI subprocesses** and the **whole system exposed via MCP** so any agent can drive it programmatically. Everything runs on your machine. No cloud account. No telemetry.

> **Built for the Agentic Era.** The notebook tools we love were designed for a pre-AI world. Arima keeps the best of that lineage — cells, kernels, rich output, pipelines — and adds what the modern era demands: **your favourite AI CLI inside the notebook**, **MCP exposing every capability**, and a workflow where you can *personalize Arima to your needs in minutes*. Add a feature. Ask your CLI to package and open a PR. Ship it. *Tools should be shaped by the people who use them.*

Arima is **additive, not adversarial**. If you already love another notebook tool, keep using it. Arima reads and writes `.ipynb` alongside its native `.vnb` format — so you can prototype here, export there, and back, freely. Use what fits the moment; pick the canvas that gives you the best leverage.

What's inside this brochure

At a glance

- **7 languages** — JShell · Java · JS · TS · C# · F# · C++ — one UI.
- **CLI inside** — Claude · Copilot · Gemini as local subprocess.
- **MCP outside** — Whole system exposed to any agent over MCP.
- **.ipynb interop** — Round-trip with Jupyter — pick your canvas.
- **Live output** — STOMP/WebSocket streaming. Zero polling.
- **3 package mgrs** — Maven · npm · NuGet — one click each.
- **Pipeline DSL** — `//@ anchor` / `//@ depends` across all 7 modes.
- **Personalize fast** — Add features in minutes — your CLI opens the PR.

02 Capabilities & Features

Every capability in Arima is designed around two principles: *your environment, your machine*, and *fast iteration from idea to executed cell*. Here is the full feature surface.

JShell-native execution

Every JShell cell shares one persistent session per notebook. Variables, imports, and method definitions persist across cells exactly like a classic REPL.

Full `javac` compile mode

Switch any cell to Java mode to write a full `public class`. Compiled with `javax.tools`, line numbers normalised, stdout/stderr captured.

JavaScript & TypeScript

Node 18+ for JS; Node 22.6+ type-stripping for TS, with optional `tsc --noEmit` for full diagnostics. Both share the same npm modules.

C# & F# via .NET SDK

C# 9+ top-level programs via `dotnet run`; F# scripts via `dotnet fsi`. NuGet packages auto-injected as `<PackageReference>` or `#r "nuget:"`.

C++ with auto-detected toolchain

MSVC, GCC, or Clang — auto-detected on `PATH`. 25 standard headers pre-included. Notebook mode wraps cells in `main()` automatically.

Real-time output

STOMP over SockJS streams stdout/stderr live as the cell runs. No polling, no buffering surprises.

Maven package manager

Type `groupId:artifactId:version`, click Install. Arima downloads from Maven Central and injects the JAR into the active JShell classpath immediately.

npm package manager

One-click install for JavaScript *and* TypeScript cells. Modules land in `data/npm-modules/node_modules/`; `require()` resolves automatically.

NuGet package manager

Same flow for C# and F#. Installed packages are wired into every C# project file and every F# script as `#r "nuget:"` on the first line.

Pipeline orchestration

Annotate cells with `//@ anchor` and `//@ depends`. The orchestration service builds a DAG, topologically sorts, and runs ancestors before your target cell. Works in all 7 languages, including cross-notebook references.

Multi-provider AI

Claude, GitHub Copilot, and Gemini all run as local CLI subprocesses. Switch providers from the AI tab. No API keys stored by Arima; provider auth lives with the CLI.

AI language conversion

Switch a cell from Java to C#, or from JS to TypeScript, and Arima offers to convert the existing code using the active AI provider. Same intent, new dialect.

MCP server

Expose Arima itself as an MCP tool server — Claude Code, Claude Desktop, and custom agents can drive cells, manage packages, and

Variable inspector

Inspect live variable state across JShell, JS, TS, C#, F#, and C++ sessions. See types, values, and structure without printing.

03 Seven languages, one canvas

Each cell carries a mode badge that mirrors the language colour shown in the live UI. Switch modes from the badge button at the top of any cell.

JShell — shared session

Every JShell cell in the notebook shares one `jdk.jshell` instance. Pre-loaded imports for `java.util`, `java.time`, XChart, Tablesaw, Commons Math, and the `BaristaDisplay` helpers. Variables and methods persist across cells until you press **Restart**.

Java — full classes

Switch a cell to Java mode for a complete `public class Main` with `main()`. Compiled via `javax.tools`, errors are line-number-normalised, stdout/stderr captured from the subprocess.

JavaScript — Node.js

Runs in a `node -e` subprocess. `require()` resolves from `data/npm-modules`. `barista.table()`, `barista.html()`, `barista.display()`, `barista.stats()` are available globally.

TypeScript — Node 22.6+

Cells run via Node.js's built-in `--experimental-strip-types`. Install `typescript` globally to enable `tsc --noEmit` diagnostics with proper line numbers. Same helpers as JS, full type signatures.

C# — `dotnet run`

Cells are wrapped as a C# 9+ top-level program inside a temp `.csproj`. NuGet packages auto-injected as `<PackageReference>`. Type declarations are moved to the end automatically to satisfy CS8803.

F# — `dotnet fsi`

Runs as an `.fsx` script. Inline `#r "nuget:"` directives are extracted and hoisted to the top of the file so `fsi` resolves them before compilation. `System`, `System.Linq`, and generics are pre-opened.

C++ — MSVC / GCC / Clang

Compiler auto-detected on `PATH` with `-std=c++17 -Wall`. 25 standard headers pre-included. Notebook mode wraps in `main()` automatically; if your cell declares `int main(` Arima treats it as a complete program. Declarations are split to global scope, statements stay in `main()`. Helpers: `baristaHtml`, `baristaDisplay`, `baristaTable`.

Cross-language pipelines

The `//@ anchor` / `//@ depends` DSL is identical across every runtime. The orchestration service builds the DAG once and dispatches each step to its language-specific execution service.

Example — pipeline across cells

```
//@ anchor: load-data
//@ description: Load CSV from disk (JShell)
var data = Table.read().csv("data.csv");

//@ anchor: clean-data
//@ depends: load-data
var clean = data.dropRowsWithMissingValues();

//@ anchor: visualize
```

04 Architecture at a glance

Arima is a single Spring Boot 3 application on Java 21. The browser loads static HTML/CSS/JS directly from Spring's static resource handler — there is no frontend build step. REST and STOMP endpoints sit in front of a fleet of execution services, each owning one language runtime.

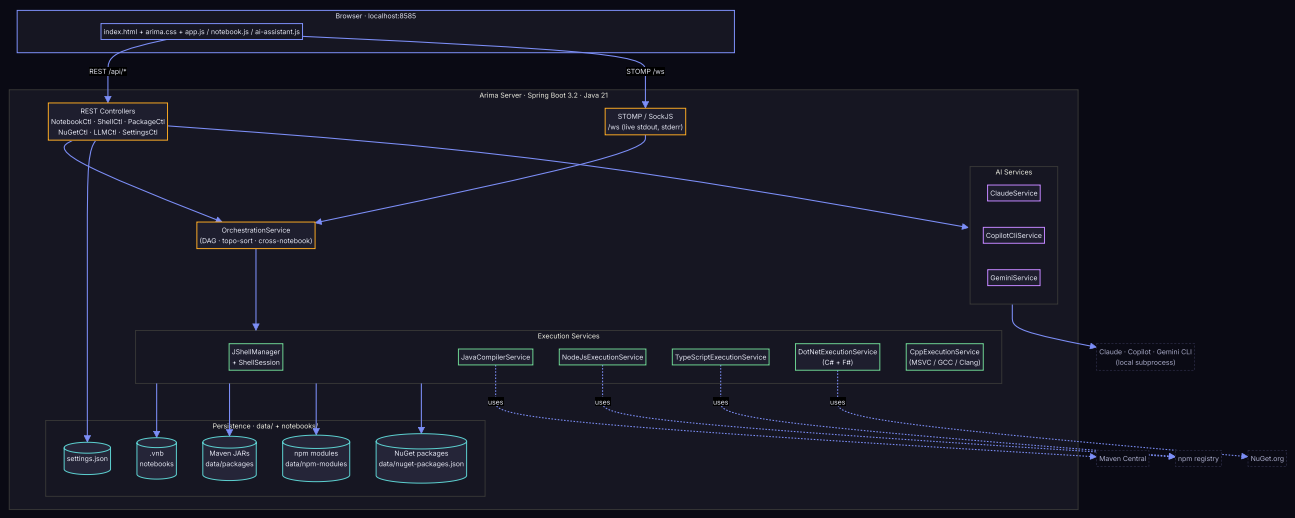

Figure 1 — Component map. Browser ↔ Spring Boot ↔ language-specific execution services ↔ local disk & package registries.

Execution flow for a single cell

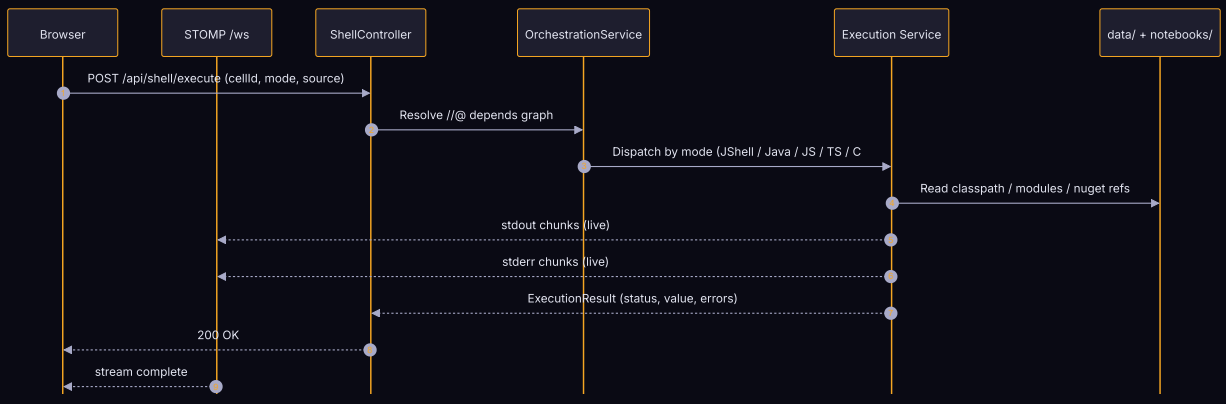

Figure 2 — One cell, end-to-end. Real-time output streams over STOMP while the REST request stays open for the final result.

05 Install & quick start

Arima is a single Java app. There is one JAR, one config file, and one port. The `arima` launcher detects what is missing, builds the JAR if needed, starts the server, and opens your browser.

Prerequisites

Tool	Version	Why
JDK	17+ (21 recommended)	JShell + Spring Boot runtime. Must be a JDK, not just JRE.
Maven	3.8+	Build from source. Skip if you only run a published JAR.
Node.js	18+ (22.6+ for TS)	JavaScript / TypeScript cells + npm packages.
.NET SDK	6.0+	C# (`dotnet run`) and F# (`dotnet fsi`).
C++ compiler	any	MSVC on Windows, GCC or Clang on Linux/macOS — auto-detected.
AI CLI	latest	Optional. Claude CLI / GitHub Copilot CLI / Gemini CLI.

Five-step quick start

1 Clone the repo
`git clone https://github.com/snchande/arima-notebooks.git && cd arima-notebooks`

2 Start with the arima CLI
Windows CMD: `arima` · PowerShell: `./arima.ps1` · Linux/macOS: `./arima.sh`

3 Open the browser
The launcher opens `http://localhost:8585` automatically. The first build downloads Maven deps.

4 Create your first notebook
Click the folder icon → **+ New Notebook**. Name it. A starter code cell opens immediately.

5 (Optional) Enable AI
Install a CLI: `claude`, `github-copilot-cli`, or `gemini`. Pick the provider in **Settings → AI Provider**. No API keys touch Arima. `arima stop` · `arima status` · `arima rebuild` manage the lifecycle.

Running the JAR directly

```
java --add-opens=jdk.jshell/jdk.jshell=ALL-UNNAMED \
     --add-opens=java.base/java.lang=ALL-UNNAMED \
     --add-exports=jdk.jshell/jdk.jshell=ALL-UNNAMED \
     -jar target/arima-notebooks-1.0.0-SNAPSHOT.jar
```

06 Your first notebook

Arima opens with a single empty code cell in JShell mode. From here, you can switch modes, install packages, chain dependencies, and ask an AI to fill in the body.

JShell mode — shared state across cells

```
// Cell 1 (JShell)
var name = "Arima";
var version = 1.2;

// Cell 2 (JShell) — name and version are already in scope
System.out.printf("Hello from %s v%.1f%n", name, version);
```

Java mode — full class compilation

```
public class Fibonacci {
  public static void main(String[] args) {
    int a = 0, b = 1;
    for (int i = 0; i < 10; i++) {
      System.out.print(a + " ");
      int t = a + b; a = b; b = t;
    }
  }
}
```

Installing a Maven package

Open the **Packages** tab → enter `com.google.code.gson:gson:2.10.1` → click **Install**. The JAR downloads from Maven Central and is added to the active JShell classpath in seconds — no restart required.

Pipeline orchestration — cells with dependencies

```
//@ anchor: parse
//@ description: Parse the JSON input
var json = "{\"x\": 42}";
var obj  = new Gson().fromJson(json, Map.class);

//@ anchor: render
//@ depends: parse
barista.display(obj);
```

Click **Run with Dependencies** on the *render* cell, and Arima automatically runs *parse* first, then *render* — in topological order. The same syntax works in C#, F#, C++, JS, and TS.

Keyboard shortcuts

`Ctrl+Enter`	Run current cell
`Shift+Enter`	Run cell and move focus to next
`Ctrl+\`	Toggle AI panel

07 The Agentic Era — CLI inside, MCP outside

Arima treats AI as **first-class infrastructure**, not a side panel. Three things make this real: a CLI runs *inside* every notebook session as a local subprocess; the entire Arima capability surface is *outside*-exposed over MCP so any agent can drive it; and you can personalize Arima itself the same way — by asking the same CLI to write the change and open the PR. No API keys live with Arima. Authentication stays in the CLI you trust.

Claude

Install: claude.ai/code → run `claude auth`. Used via the `claude` binary on PATH.

GitHub Copilot

`npm install -g @githubnext/github-copilot-cli` → `github-copilot-cli auth`.

Gemini

`npm install -g @google/gemini-cli` → `gemini auth`.

What you can ask the assistant

- *"Write a C++ cell that sorts a vector using std::sort and prints the result."*
- *"Explain what this cell does"* — attach a cell with the 🤖 button.
- *"Convert this Java cell to C#."* — Arima offers conversion automatically when you switch a cell's mode.
- *"Generate a notebook showing Java streams with examples."* — produces a full `.vnb` file.
- *"Why did this cell fail?"* — the failed cell's stderr is attached automatically.

Arima as an MCP server — the whole system, exposed

Pick the canvas that fits the moment. Drive Arima from your terminal, from Claude Code or Claude Desktop, from a custom agent — or from Arima itself. Same capability surface, three ways in.

List notebooks	Read directory of `.vnb` files including tutorials.
Read & write cells	Append, modify, and reorder cells programmatically.
Execute cells	Run any cell in any of the seven modes; get the full execution result.
Manage packages	Install Maven, npm, or NuGet packages on behalf of an agent.
Inspect state	Query live variables in JShell, JS, TS, C#, F#, and C++ sessions.
Import / export `.ipynb`	Round-trip between Arima and Jupyter so you pick the canvas, not the tool.

Personalize Arima in minutes. If a capability is missing, the workflow is the point: ask your CLI to add the feature — service, controller, UI hook — verify with `arima rebuild`, and ask the CLI again to package the change as a PR. New capability in less than an hour. If it helps you, it probably helps others; ship it back.

08 Personalize & contribute

Arima is meant to be **shaped by the people who use it**. The loop is short: clone, describe the change to your AI CLI, verify, ask the same CLI to package the PR. The repo ships with explicit instruction files for Claude, Copilot, and Gemini so the agent respects the architecture while it works.

1 — Fork & clone

```
git clone https://github.com/<your-fork>/Arima.git
cd arima-notebooks
git remote add upstream https://github.com/snchande/arima-notebooks.git
git checkout -b feat/<short-name>
```

2 — Pick your AI co-pilot

Claude CLI

Run `claude` in the repo root. It reads `CLAUDE.md` and `AGENTS.md` automatically — both encode the architecture rules.

GitHub Copilot

Inline in VS Code or via `gh copilot`. Reads `.github/copilot-instructions.md` — same architecture rules.

Gemini CLI

Run `gemini` in the repo root. Reads `GEMINI.md` — same architecture rules restated for Gemini's prompt format.

3 — Describe the feature, let the CLI write it

Open your AI CLI in the repo root and describe the capability you want. The agent files from step 2 keep it inside the layered architecture: *model → service → controller → static UI*. No layer-merging, no smuggling backends into the frontend, no surprise network calls.

4 — Verify locally

```
arima rebuild
arima start
# exercise your feature in the browser; check WebSocket output
mvn test
pwsh ./scripts/security-check.ps1   # or .sh on Linux/macOS
```

5 — Ask your CLI to package and PR

Same CLI, one more prompt: *"package these changes as a PR against `master`, fill the PR template, attach screenshots if UI."* Push, open, done. If the change personalizes Arima only for you, keep it local. If it helps others, ship it back.

6 — Wait for the green checks

Every PR triggers GitHub Actions checks before review:

`build`	Maven compile + unit tests on JDK 21.
`security-scan`	Runs `scripts/security-check.ps1` / `.sh` — see page 10.

09 Security & quality gates

Every pull request runs through an automated security check before it can be marked **ready for review**. The intent is simple: catch obvious risk before a human spends time on it.

The PR security pipeline

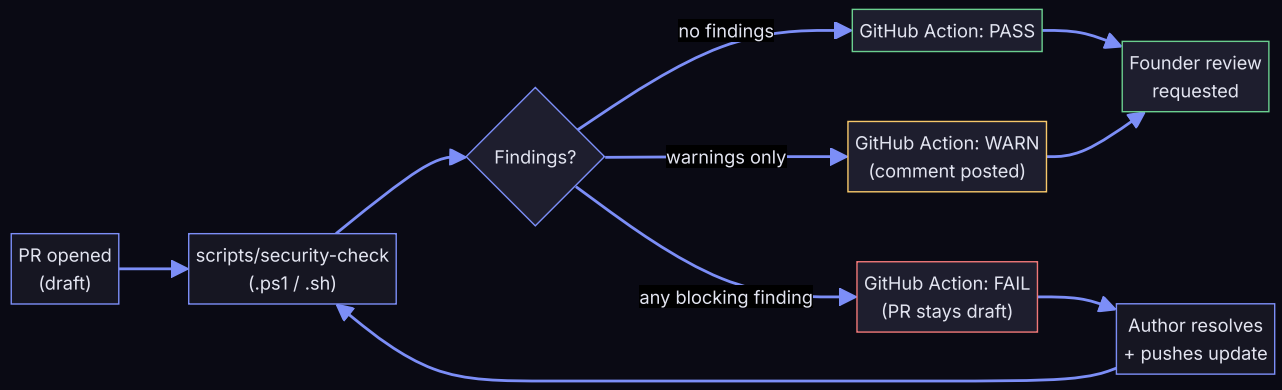

Figure 3 — A PR cannot become "ready for review" until `security-check` reports zero blocking findings.

What the script checks

Check	Why it matters
Secret patterns (`sk-…`, AWS keys, private-key PEM headers, GitHub tokens)	Prevents accidental credential leaks. Blocks the PR.
Hard-coded API keys in `data/settings.json`	The settings file is `.gitignore`-d for a reason. Blocks.
Calls to `Runtime.exec` with user-controlled strings	Command-injection risk in execution services. Blocks.
New outbound HTTP hosts (anything beyond Maven Central, npm, NuGet, AI CLIs)	Preserves the local-first guarantee. Blocks unless allow-listed.
Dependency vulnerabilities (`mvn dependency:check`)	Known-CVE detection. High-severity blocks; medium warns.
Architecture lint — layer crossings (frontend → DB, controller → controller)	Preserves the contract in `AGENTS.md`. Blocks.
Test coverage for changed files (best-effort)	Warns if a new service has no companion test.

Local pre-flight

```
# Windows
pwsh ./scripts/security-check.ps1

# Linux / macOS
```

10 About & contact

Arima Notebooks is an open-source product distributed under the **MIT License**. It is built and maintained by **Suresh Chande** with help from the community — and with a healthy population of AI co-pilots, which is exactly the workflow Arima is built to support. This project stands on the shoulders of every great notebook tool that came before it; the goal is to *add* to that lineage, not replace it.

Suresh Chande

Founding contributor · Maintainer

✉ suresh.chande@gmail.com

↗ github.com/snchande/arima-notebooks

How to get involved

File an issue

Bugs, feature requests, tutorial ideas — anything. Use the issue templates under `.github/ISSUE_TEMPLATE`.

Open a discussion

Got an idea but not sure where it fits? Start a GitHub Discussion. The maintainer reads them all.

Send a PR

Follow the *Contributor Playbook* on page 09. Small, focused PRs are merged fastest.

Share a tutorial

Ship a `.vnb` file to the tutorial library. New languages, new techniques — all welcome.

Licence & attribution

Arima Notebooks is released under the **MIT License**. You may use, modify, and distribute it freely, in personal and commercial projects, as long as the copyright notice is retained. JShell, Java, the .NET SDK, and the AI provider CLIs remain the property of their respective owners — Arima orchestrates them; it does not redistribute them.

Thanks. If Arima saves you time, the simplest way to say thanks is a star on the repo and a PR — even a one-line typo fix lands you on the contributor list.